

MSP — Multi-Party Signature Protocol

Protocolo de Acordo com Múltiplas Assinaturas Digitais

Guilherme Ricci

Universidade Federal de Santa Catarina – UFSC

4 de setembro de 2026

Resumo

Este relatório apresenta o desenvolvimento do MSP (*Multi-Party Signature Protocol*), um protótipo desenvolvido em C++ para criação e verificação de acordos digitais envolvendo múltiplos participantes.

O protocolo utiliza assinaturas digitais RSA, certificados X.509 e estruturas CMS/PKCS#7 do tipo *SignedData*. Além do documento, o conteúdo assinado inclui uma política que define os participantes esperados e as condições necessárias para conclusão do acordo.

O projeto utiliza a biblioteca LibCryptoSEC, desenvolvida pelo Laboratório de Segurança em Computação da Universidade Federal de Santa Catarina, em conjunto com o OpenSSL.

Sumário

1	Introdução	3
1.1	Motivação	3
1.2	Objetivos	3
2	Fundamentação Teórica	4
2.1	Funções de resumo criptográfico	4
2.2	Assinaturas digitais	4
2.3	Certificados X.509	5
2.4	CMS e PKCS#7	5
3	Arquitetura do MSP	5
3.1	Operator	5
3.2	Agreement	6
3.3	MultiSignature	6

4	Implementação	6
4.1	Geração das chaves	6
4.2	Política do acordo	7
5	Análise de Segurança	7
5.1	Integridade	7
5.2	Participantes	7
5.3	Modelo de confiança	8
5.4	Limitações	8
6	Testes	8
7	Resultados e Discussão	9
8	Trabalhos Futuros	9
9	Conclusão	10

1 Introdução

Assinaturas digitais permitem verificar a integridade de informações e comprovar que determinada operação criptográfica foi realizada utilizando uma chave privada.

Em determinados sistemas, entretanto, uma operação não deve ser autorizada por apenas um participante. Contratos, documentos e operações administrativas podem exigir a aprovação de múltiplas entidades antes de serem considerados válidos.

O projeto MSP (*Multi-Party Signature Protocol*) investiga esse problema através da implementação de um protocolo de acordo baseado em coassinaturas CMS/PKCS#7.

O protótipo foi desenvolvido em C++98 utilizando a biblioteca LibCryptoSEC e o OpenSSL.

1.1 Motivação

O desenvolvimento do MSP foi motivado pelo estudo de criptografia aplicada, certificados digitais, Infraestruturas de Chaves Públicas e sistemas que exigem autorização de múltiplas entidades.

O projeto busca explorar principalmente:

- assinaturas digitais;
- identificação de signatários;
- certificados X.509;
- estruturas CMS/PKCS#7;
- políticas de aprovação;
- modelos de confiança;
- validação de integridade.

1.2 Objetivos

O objetivo geral deste trabalho é implementar um protocolo capaz de representar um acordo digital envolvendo múltiplos signatários.

Os objetivos específicos são:

- criar identidades criptográficas para os participantes;
- gerar pares de chaves RSA;
- emitir certificados X.509 para os operadores;

- produzir múltiplas assinaturas sobre o mesmo conteúdo;
- armazenar as assinaturas em uma estrutura CMS *SignedData*;
- verificar os participantes que efetivamente assinaram;
- detectar modificações no conteúdo assinado;
- analisar as limitações do modelo de confiança utilizado.

2 Fundamentação Teórica

2.1 Funções de resumo criptográfico

Uma função de resumo criptográfico transforma uma entrada de tamanho arbitrário em uma saída de tamanho fixo.

Podemos representar uma função de resumo por:

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n \quad (1)$$

Para uma mensagem m , o resumo correspondente é dado por:

$$h = H(m) \quad (2)$$

O MSP utiliza o algoritmo SHA-256.

2.2 Assinaturas digitais

Um esquema de assinatura digital pode ser representado de forma simplificada pelas seguintes operações:

$$(pk, sk) \leftarrow \text{KeyGen}() \quad (3)$$

$$\sigma \leftarrow \text{Sign}(sk, m) \quad (4)$$

$$b \leftarrow \text{Verify}(pk, m, \sigma) \quad (5)$$

onde sk representa a chave privada, pk representa a chave pública, m representa a mensagem e σ representa a assinatura.

No protótipo atual, os operadores utilizam chaves RSA de 2048 bits.

2.3 Certificados X.509

Certificados X.509 associam uma chave pública a informações de identificação de uma entidade.

No MSP, cada operador possui um certificado contendo seu identificador.

Atualmente, os certificados utilizados no protótipo são autoassinados. Essa escolha simplifica o ambiente experimental, mas não estabelece uma cadeia de confiança externa.

2.4 CMS e PKCS#7

O CMS (*Cryptographic Message Syntax*) permite representar estruturas contendo conteúdo assinado digitalmente.

O MSP utiliza uma estrutura *SignedData*, que contém:

- o conteúdo assinado;
- os certificados dos participantes;
- informações sobre os algoritmos utilizados;
- estruturas *SignerInfo*;
- as assinaturas digitais.

É importante diferenciar os certificados presentes no pacote das estruturas *SignerInfo*. A presença de um certificado não significa, por si só, que aquele participante produziu uma assinatura.

3 Arquitetura do MSP

A arquitetura atual do MSP pode ser dividida em três componentes principais:

- `Operator`;
- `Agreement`;
- `MultiSignature`.

3.1 Operator

A classe `Operator` representa um participante do protocolo.

Cada operador possui:

- nome;

- identificador;
- endereço de e-mail;
- par de chaves RSA;
- certificado X.509.

3.2 Agreement

A classe **Agreement** representa uma execução do protocolo de acordo.

O acordo possui quatro possíveis estados:

$$S = \{\text{PENDING}, \text{COMPLETE}, \text{ABORTED}, \text{FINALIZED}\} \quad (6)$$

Quando todos os participantes esperados assinam:

$$\text{PENDING} \rightarrow \text{COMPLETE} \rightarrow \text{FINALIZED}$$

Caso um participante rejeite o acordo:

$$\text{PENDING} \rightarrow \text{ABORTED}$$

3.3 MultiSignature

A classe **MultiSignature** encapsula a construção do pacote CMS/PKCS#7.

O primeiro participante inicializa a estrutura *SignedData*. Os demais são posteriormente adicionados como novos signatários.

Ao final do processo, o conteúdo é codificado no formato DER.

4 Implementação

O MSP foi desenvolvido em C++98 e utiliza a LibCryptoSEC como uma abstração orientada a objetos das funcionalidades criptográficas fornecidas pelo OpenSSL.

4.1 Geração das chaves

Cada operador recebe um par de chaves RSA de 2048 bits.

Listing 1: Geração do par de chaves RSA.

```

1 RSAKeyPair* OperatorCreation::createOperatorKey()
2 {
3     return new RSAKeyPair(OPERATOR_KEY_SIZE);

```

4.2 Política do acordo

Antes da criação das assinaturas, o MSP gera uma representação canônica da política do protocolo.

```
MSP-POLICY/1
mode=unanimous
required=3
signer=operator-1
signer=operator-2
signer=operator-3
--PDF--
<documento>
```

Dessa forma, os participantes não assinam apenas o PDF. As condições do acordo também fazem parte do conteúdo autenticado criptograficamente.

5 Análise de Segurança

A validade matemática de uma assinatura não é suficiente para estabelecer todas as propriedades de segurança de um protocolo.

Também é necessário determinar quais identidades são confiáveis, quais participantes são obrigatórios e quais informações fazem parte da operação assinada.

5.1 Integridade

Uma modificação no conteúdo coberto pela assinatura deve provocar falha na verificação.

Se m representa o conteúdo original e m' uma versão adulterada:

$$m \neq m' \tag{7}$$

uma assinatura produzida sobre m não deve ser considerada válida para m' .

5.2 Participantes

Na política atual, três operadores devem participar:

$$S_{\text{esperados}} = \{\text{operator-1}, \text{operator-2}, \text{operator-3}\} \tag{8}$$

Para que um acordo seja aceito, deve valer:

$$S_{\text{verificados}} = S_{\text{esperados}} \quad (9)$$

O MSP verifica os registros *SignerInfo* existentes no pacote CMS em vez de considerar apenas os certificados nele armazenados.

5.3 Modelo de confiança

Como os certificados do protótipo são autoassinados, o MSP utiliza atualmente fingerprints SHA-256 como âncoras de confiança.

Uma representação simplificada do arquivo utilizado é:

```
MSP-TRUST/1
operator-1=<SHA-256>
operator-2=<SHA-256>
operator-3=<SHA-256>
```

Esse mecanismo permite vincular cada identificador a um certificado esperado.

Entretanto, o arquivo de confiança precisa ser distribuído e armazenado de forma segura.

5.4 Limitações

A implementação atual ainda não possui:

- uma Autoridade Certificadora própria;
- validação completa de cadeias X.509;
- CRL ou OCSP;
- armazenamento das chaves privadas em HSM;
- políticas gerais do tipo *t-de-n*;
- criptografia de limiar.

6 Testes

O projeto utiliza cenários positivos e negativos para verificar o comportamento do protocolo.

Tabela 1: Principais cenários de teste do MSP.

Cenário	Entrada	Resultado
Acordo completo	3 assinaturas	Aceito
Recusa	operator-2 rejeita	Abortado
Pacote alterado	DER modificado	Rejeitado
Trust alterado	Fingerprint inválido	Rejeitado

Os testes podem ser executados com:

```
make test
```

7 Resultados e Discussão

A implementação demonstra que um acordo envolvendo múltiplos participantes pode ser representado utilizando múltiplas estruturas *SignerInfo* dentro de um mesmo CMS *SignedData*.

Durante o desenvolvimento tornou-se importante separar três propriedades:

1. integridade do conteúdo;
2. identificação dos participantes que efetivamente assinaram;
3. cumprimento da política de aprovação.

A implementação atual atende a essas propriedades dentro das hipóteses estabelecidas para o protótipo.

Entretanto, certificados autoassinados e o uso de fingerprints externos ainda limitam o modelo de confiança.

8 Trabalhos Futuros

As próximas evoluções planejadas para o MSP incluem:

- implementação de uma pequena Infraestrutura de Chaves Públicas;
- emissão de certificados por uma Autoridade Certificadora;
- validação de cadeias X.509;
- suporte a revogação;
- integração com PKCS#11;

- armazenamento de chaves em HSM;
- políticas de aprovação t -de- n ;
- registro de auditoria das operações.

9 Conclusão

Este trabalho apresentou o MSP, um protótipo para estudo de acordos digitais envolvendo múltiplos participantes.

O desenvolvimento permitiu aplicar conceitos relacionados a funções de resumo criptográfico, RSA, certificados X.509, CMS/PKCS#7 e políticas de autorização.

Além da geração das assinaturas, o projeto evidencia a importância de distinguir a validade criptográfica de uma assinatura da confiança depositada na identidade correspondente.

Como evolução, pretende-se aproximar o modelo de confiança do MSP de uma Infraestrutura de Chaves Públicas completa, incluindo uma Autoridade Certificadora, mecanismos de revogação e armazenamento seguro das chaves privadas.

Referências

- [1] R. Housley. *Cryptographic Message Syntax (CMS)*. RFC 5652, Internet Engineering Task Force, 2009.
- [2] OpenSSL Project. *OpenSSL Cryptography and SSL/TLS Toolkit*. Disponível em: <https://www.openssl.org/>.
- [3] LabSEC. *LibCryptoSEC*. Disponível em: <https://github.com/LabSEC/libcryptosec>.
- [4] Guilherme Ricci. *MSP — Multi-Party Signature Protocol*. Disponível em: <https://github.com/riccizzx/msp>.